

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 12: Intractability: NP-Completeness and Reductions

Foreign Trade University

Contents

1	Introduction: When Cleverness Runs Out	1
1.1	What this week is really about	1
2	Polynomial-Time Reductions	2
2.1	Definition	2
2.2	A first example: Independent Set and Vertex Cover	2
2.3	Independent Set and Clique	3
3	The Classes P and NP	3
3.1	Decision problems and encodings	3
3.2	Certificates and verifiers	3
3.3	The P vs. NP question	4
4	NP-Completeness and Cook–Levin	4
4.1	NP-hardness and NP-completeness	4
4.2	The Cook–Levin theorem	4
5	The Standard Web of Reductions	5
5.1	3-SAT $\leq_p$ Independent Set	5
5.2	Independent Set $\leq_p$ Vertex Cover $\leq_p$ Clique	6
5.3	Hamiltonian Cycle and the Traveling Salesman	6
5.4	Subset Sum	7
5.5	Graph Coloring	7
6	The Structure of an NP-Completeness Proof	7
6.1	A common pitfall: reducing the wrong way	8
7	Summary and Looking Ahead	8

1 Introduction: When Cleverness Runs Out

For eleven weeks we have built a toolkit for designing *efficient* algorithms: greedy strategies, divide-and-conquer, dynamic programming, and network flow. Each gave us polynomial-time solutions to problems that, at first glance, looked like they might require searching exponentially many possibilities.

This week we confront the opposite phenomenon. For an enormous and important collection of problems – satisfiability of logical formulas, finding large cliques in graphs, packing a knapsack

exactly, routing a salesman through cities – *nobody has ever found a polynomial-time algorithm*, despite decades of effort by many brilliant people. The theory of **NP-completeness** (Kleinberg & Tardos, Ch. 8) explains why: these problems are all, in a precise technical sense, *equally hard*. A polynomial-time algorithm for any one of them would yield one for all of them, and would resolve the most famous open problem in computer science, P vs. NP.

The central technique is the **polynomial-time reduction**: a way to transform one problem into another so efficiently that solving the second solves the first. Reductions let us *spread hardness*: once we know a single problem is hard, we can show many others are hard by reducing the known-hard problem to them. Crucially, this is a *positive* skill, not just a pessimistic one. Recognizing that a problem is NP-complete tells you to stop searching for an exact polynomial algorithm and instead reach for the tools of Weeks 13–14: exponential algorithms with good constants, fixed-parameter tractability, approximation, and heuristics.

1.1 What this week is really about

Three intertwined ideas:

- **Verification vs. search.** For all the problems we study, a proposed solution can be *checked* quickly even though *finding* one seems to require exponential search. This asymmetry is the definition of the class NP.
- **Reduction.** Problem X *reduces* to problem Y ($X \leq_p Y$) if an efficient algorithm for Y could be used as a subroutine to solve X efficiently. We use reductions to argue “ Y is at least as hard as X .”
- **Completeness.** A problem is *NP-complete* if it is in NP *and* every problem in NP reduces to it. These are the hardest problems in NP – and the Cook–Levin theorem proves they exist.

2 Polynomial-Time Reductions

2.1 Definition

Definition 2.1 (Polynomial-time reduction). *We say problem X reduces in polynomial time to problem Y , written $X \leq_p Y$, if an arbitrary instance of X can be solved using a polynomial number of standard computational steps plus a polynomial number of calls to a (hypothetical) subroutine, or oracle, that solves Y .*

Intuitively, $X \leq_p Y$ means “ Y is at least as hard as X ”: if we could solve Y efficiently, we could solve X efficiently too. Two immediate consequences make reductions the workhorse of this chapter.

Proposition 2.2 (Spreading tractability and intractability). *Suppose $X \leq_p Y$.*

- (a) *If Y can be solved in polynomial time, then so can X .*
- (b) *If X cannot be solved in polynomial time, then neither can Y .*

Proof. (a) Run the reduction: a polynomial number of steps, each oracle call replaced by the polynomial-time algorithm for Y . A polynomial number of polynomial-time calls, composed with polynomial overhead, is polynomial. (b) is the contrapositive of (a). $\square$

Part (b) is how we prove hardness; part (a) is the (often overlooked) reason reductions are also a constructive design tool – if you can reduce your new problem to a problem you *can* already solve, you are done.

2.2 A first example: Independent Set and Vertex Cover

Two graph problems will recur throughout the week. Fix a graph $G = (V, E)$.

Definition 2.3 (Independent Set). *A set $S \subseteq V$ is independent if no two vertices in S are joined by an edge. The INDEPENDENT SET decision problem asks: given G and an integer k , does G contain an independent set of size at least k ?*

Definition 2.4 (Vertex Cover). *A set $S \subseteq V$ is a vertex cover if every edge of G has at least one endpoint in S . The VERTEX COVER decision problem asks: given G and an integer k , does G contain a vertex cover of size at most k ?*

These two problems are, in a precise sense, the same problem viewed from opposite sides.

Lemma 2.5 (Complementarity). *For a graph $G = (V, E)$ with $|V| = n$, a set $S \subseteq V$ is an independent set if and only if its complement $V \setminus S$ is a vertex cover. Consequently, G has an independent set of size $\geq k$ iff it has a vertex cover of size $\leq n - k$.*

Proof. Suppose S is independent. Take any edge $(u, v) \in E$. Since S is independent, u and v cannot both lie in S , so at least one of them lies in $V \setminus S$ – hence $V \setminus S$ covers every edge. Conversely, if $V \setminus S$ is a vertex cover, then no edge has both endpoints in S (otherwise that edge would be uncovered), so S is independent. The size statement follows since $|V \setminus S| = n - |S|$. $\square$

Lemma 2.5 gives reductions *in both directions*: INDEPENDENT SET $\leq_p$ VERTEX COVER and VERTEX COVER $\leq_p$ INDEPENDENT SET. The reduction is trivial – leave G alone and replace k by $n - k$ – yet it already shows the two problems are computationally equivalent. Problem 4 asks you to verify this complementarity computationally, and Problem 12 builds the explicit bidirectional mapping.

2.3 Independent Set and Clique

Definition 2.6 (Clique). *A set $S \subseteq V$ is a clique if every two vertices in S are joined by an edge. The CLIQUE problem asks whether G has a clique of size $\geq k$.*

Lemma 2.7. *S is an independent set in G if and only if S is a clique in the complement graph $\overline{G}$ (the graph on the same vertices whose edges are exactly the non-edges of G). Hence INDEPENDENT SET $\leq_p$ CLIQUE (map G to $\overline{G}$, keep k).*

Proof. S independent in G means no pair in S is an edge of G , which means every pair in S is an edge of $\overline{G}$, i.e. S is a clique in $\overline{G}$. The construction of $\overline{G}$ takes $O(n^2)$ time, so this is a polynomial reduction. $\square$

Problem 5 asks you to confirm experimentally that the maximum clique of G equals the maximum independent set of $\overline{G}$.

3 The Classes P and NP

3.1 Decision problems and encodings

To talk precisely about “efficient” we restrict attention to **decision problems**: problems with a yes/no answer. This is not a real restriction – an optimization problem (“find the largest independent set”) is essentially as hard as its decision version (“is there an independent set of size $\geq k$?”), since we can binary-search on k . We think of a problem X as the set of input strings s (over some finite alphabet) for which the answer is “yes,” and write $X(s) = \text{yes}$ accordingly.

Definition 3.1 (The class P). *P is the set of decision problems X for which there is an algorithm A and a polynomial $p(\cdot)$ such that for every input s , $A(s)$ halts within $p(|s|)$ steps and outputs the correct answer $X(s)$.*

3.2 Certificates and verifiers

The defining feature of the problems in this chapter is that, although *finding* a solution seems hard, *checking* a proposed solution is easy. We formalize “proposed solution” as a *certificate*.

Definition 3.2 (Efficient verifier). *A verifier for a problem X is an algorithm $B(s, t)$ taking an input string s and a certificate string t . We say B is an efficient verifier if it runs in polynomial time in $|s|$ and there is a polynomial p such that for every string s :*

$$X(s) = \text{yes} \iff \text{there exists a string } t \text{ with } |t| \leq p(|s|) \text{ such that } B(s, t) = \text{yes}.$$

In words: an input is a “yes” instance exactly when there is a short certificate that the verifier accepts. The verifier does not have to *find* t ; it only has to *recognize* a valid one.

Definition 3.3 (The class NP). *NP is the set of decision problems for which there exists an efficient verifier.*

Example 3.4. *For INDEPENDENT SET, the certificate for input (G, k) is a set S of k vertices; the verifier checks (in polynomial time) that $|S| \geq k$ and that no edge of G has both endpoints in S . Problem 3 implements exactly this verifier. For 3-SAT, the certificate is a satisfying assignment, and the verifier evaluates the formula – Problem 1.*

Theorem 3.5. $P \subseteq NP$.

Proof. If $X \in P$ via a polynomial-time algorithm A , define the verifier $B(s, t) := A(s)$, ignoring the certificate t entirely (use $t = \text{empty string}$). Then B runs in polynomial time, and $X(s) = \text{yes}$ iff there exists a (trivial) certificate making B accept, namely whenever $A(s) = \text{yes}$. Hence $X \in NP$. $\square$

3.3 The P vs. NP question

The name NP stands for “nondeterministic polynomial time”: equivalently, a problem is in NP if a nondeterministic machine could *guess* the certificate and then verify it in polynomial time. We know $P \subseteq NP$. The trillion-dollar question is whether the inclusion is strict:

Is $P = NP$?

That is, does the ability to *verify* solutions efficiently imply the ability to *find* them efficiently? Almost everyone believes $P \neq NP$ – that verification really is easier than search – but no proof is known in either direction. The machinery of NP-completeness was developed precisely to study this question.

4 NP-Completeness and Cook–Levin

4.1 NP-hardness and NP-completeness

Definition 4.1 (NP-hard, NP-complete). *A problem Y is NP-hard if $X \leq_p Y$ for every problem $X \in \text{NP}$. A problem is NP-complete if it is both NP-hard and a member of NP.*

The NP-complete problems are the “hardest” problems in NP: a polynomial-time algorithm for any single one of them would, by Proposition 2.2(a), give a polynomial-time algorithm for *every* problem in NP, proving $P = \text{NP}$.

Proposition 4.2. *If Y is NP-complete, then $Y \in P$ if and only if $P = \text{NP}$.*

Proof. If $P = \text{NP}$ then $Y \in \text{NP} = P$. Conversely, if $Y \in P$, then for every $X \in \text{NP}$ we have $X \leq_p Y$ (since Y is NP-hard), so by Proposition 2.2(a), $X \in P$; hence $\text{NP} \subseteq P$, and with $P \subseteq \text{NP}$ we get equality. $\square$

4.2 The Cook–Levin theorem

The definition of NP-completeness quantifies over *all* problems in NP, so it is not obvious that any NP-complete problem exists at all. The foundational result of the field, proved independently by Stephen Cook (1971) and Leonid Levin (1973), is that one does.

Theorem 4.3 (Cook–Levin). *SATISFIABILITY (SAT) – given a Boolean formula in conjunctive normal form, is there an assignment of truth values to the variables making it true? – is NP-complete. The same holds for 3-SAT, the special case where every clause has exactly three literals.*

Idea of the proof. Membership in NP is easy: a satisfying assignment is a certificate that a verifier can check in linear time (Problem 1). The hard direction is NP-hardness. Take any problem $X \in \text{NP}$, witnessed by an efficient verifier $B(s, t)$ running in time $p(|s|)$. The proof encodes the entire computation of B on input s – the contents of its memory at every time step – as Boolean variables, and writes down clauses that are simultaneously satisfiable exactly when there exists a certificate t causing B to accept. The number of variables and clauses is polynomial in $p(|s|)$. Thus “ s is a yes instance of X ” becomes “this CNF formula is satisfiable,” giving $X \leq_p \text{SAT}$ for arbitrary $X \in \text{NP}$. A further gadget reduction converts general clauses to 3-literal clauses, yielding NP-completeness of 3-SAT. The full construction is in Kleinberg & Tardos, Section 8.3. $\square$

Cook–Levin gives us the first NP-complete problem “from scratch.” Every subsequent NP-completeness proof gets to be far easier, thanks to the following transitivity.

Lemma 4.4 (Transitivity of $\leq_p$). *If $X \leq_p Y$ and $Y \leq_p Z$, then $X \leq_p Z$.*

Proof. Compose the two reductions: solve X using polynomially many calls to a Y -oracle, and replace each such call by the reduction that solves Y using polynomially many calls to a Z -oracle. The result uses polynomially many Z -oracle calls with polynomial overhead. $\square$

Corollary 4.5 (The standard recipe for proving NP-completeness). *To prove that a problem Y is NP-complete, it suffices to:*

- (1) *Show $Y \in \text{NP}$ (exhibit a polynomial-time verifier / short certificate);*
- (2) *Pick a known NP-complete problem X and show $X \leq_p Y$ (build a polynomial-time map from instances of X to instances of Y preserving the yes/no answer).*

By Lemma 4.4, every problem in NP reduces to X and hence to Y , so Y is NP-hard; combined with membership, Y is NP-complete.

The art of the rest of this chapter is choosing the right known-hard X and designing the **gadget** that maps X -instances to Y -instances.

5 The Standard Web of Reductions

Starting from 3-SAT (NP-complete by Cook–Levin), the following chain of reductions establishes a whole family of NP-complete problems. This is the historical backbone of the field (Karp, 1972).

5.1 3-SAT $\leq_p$ Independent Set

Theorem 5.1. 3-SAT $\leq_p$ INDEPENDENT SET.

Proof. Given a 3-SAT formula Φ with k clauses $C_1, \dots, C_k$, each clause $C_i = (\ell_{i1} \vee \ell_{i2} \vee \ell_{i3})$, we build a graph G and set the target to k .

Construction (the gadget). For each clause C_i create a *triangle*: three vertices, one per literal $\ell_{i1}, \ell_{i2}, \ell_{i3}$, joined pairwise by edges. (Within a clause, picking one literal to satisfy the clause is the idea – the triangle forces us to pick *at most* one vertex per clause into an independent set.) In addition, add a *conflict edge* between any two vertices in *different* triangles whose literals are negations of each other (e.g. a vertex labelled x_3 and a vertex labelled $\overline{x_3}$). This graph has $3k$ vertices and is built in polynomial time.

Correctness. We claim Φ is satisfiable iff G has an independent set of size $\geq k$.

($\Rightarrow$) Given a satisfying assignment, each clause has at least one true literal; pick one such literal per clause and take the corresponding vertex. This gives k vertices, one per triangle (so no triangle edge is violated), and no two of them conflict (a true literal x_3 and a true literal $\overline{x_3}$ cannot both hold under one assignment), so no conflict edge is violated. Hence an independent set of size k .

($\Leftarrow$) Conversely, an independent set S of size k can contain at most one vertex per triangle (triangle edges), and there are exactly k triangles, so it contains *exactly* one vertex per triangle. Define an assignment by setting each chosen literal’s variable to make that literal true; the absence of conflict edges within S guarantees this is consistent (we never demand x_3 true and $\overline{x_3}$ true simultaneously). Variables not pinned down may be set arbitrarily. Every clause has its chosen literal true, so Φ is satisfied. $\square$

Problem 6 implements this gadget and verifies, on small formulas, that the formula is satisfiable iff the constructed graph has an independent set of size k .

5.2 Independent Set $\leq_p$ Vertex Cover $\leq_p$ Clique

By Lemma 2.5, INDEPENDENT SET $\leq_p$ VERTEX COVER, and by Lemma 2.7, INDEPENDENT SET $\leq_p$ CLIQUE. Chaining these with Theorem 5.1 and Lemma 4.4, all three problems are NP-hard; each is easily seen to be in NP (the certificate is the set of vertices). Therefore:

Corollary 5.2. INDEPENDENT SET, VERTEX COVER, and CLIQUE are all NP-complete.

5.3 Hamiltonian Cycle and the Traveling Salesman

Definition 5.3 (Hamiltonian Cycle). A Hamiltonian cycle in a graph is a cycle that visits every vertex exactly once. The HAMILTONIAN CYCLE problem asks whether a given graph contains one.

HAMILTONIAN CYCLE is NP-complete; the reduction $3\text{-SAT} \leq_p \text{HAMILTONIAN CYCLE}$ (or, via directed graphs, $3\text{-SAT} \leq_p \text{DIRECTED HAMILTONIAN CYCLE}$) uses an intricate gadget in which each variable is a long “two-way” path traversable left-to-right or right-to-left (encoding true/false), and clauses are nodes that must be visited from one of their literals. We state the result and explore it computationally rather than reproducing the full construction.

Theorem 5.4. HAMILTONIAN CYCLE is NP-complete.

The closely related optimization problem is the **Traveling Salesman Problem**.

Definition 5.5 (Traveling Salesman Problem, decision version). Given n cities, a distance $d(i, j)$ between each pair, and a bound D , is there a tour (a cycle visiting every city exactly once and returning to the start) of total length at most D ?

Theorem 5.6. TSP (decision version) is NP-complete.

Sketch. TSP is in NP: a tour (a permutation of the cities) is a certificate whose total length is checkable in $O(n)$ time. For hardness, reduce HAMILTONIAN CYCLE $\leq_p$ TSP: given a graph G on n vertices, create one city per vertex, set $d(i, j) = 1$ if $(i, j) \in E$ and $d(i, j) = 2$ (or ∞) otherwise, and ask for a tour of length $\leq n$. A tour of length exactly n uses only edges of G and visits every vertex once – precisely a Hamiltonian cycle. So G is Hamiltonian iff the TSP instance has a tour of length $\leq n$. $\square$

Although TSP is NP-hard, *small* instances are solvable exactly. Problem 9 implements the **Held–Karp** dynamic program, which solves TSP in $O(n^2 2^n)$ time and $O(n 2^n)$ space – exponential, but a dramatic improvement over the $\Theta(n!)$ brute force over all permutations, and a preview of the “better exponential algorithms” theme of Week 13.

5.4 Subset Sum

Definition 5.7 (Subset Sum). Given positive integers $w_1, \dots, w_n$ and a target W , is there a subset $S \subseteq \{1, \dots, n\}$ with $\sum_{i \in S} w_i = W$?

Theorem 5.8. SUBSET SUM is NP-complete.

Remark on the proof. Membership in NP is immediate: the subset S is the certificate, and summing is linear (Problem 7). NP-hardness follows from a reduction $3\text{-SAT} \leq_p \text{SUBSET SUM}$ (or, in Kleinberg & Tardos, via VERTEX COVER) in which the numbers are built in a special base so that the only way to hit the target digit-by-digit is to encode a satisfying assignment without carries. The construction is in Section 8.8. $\square$

SUBSET SUM is a famous illustration of a subtle point: the textbook $O(nW)$ dynamic program (Week 8) makes it look “easy,” but $O(nW)$ is *pseudo-polynomial* – polynomial in the numeric *value* W , not in the input *size* (the number of bits, $\approx \log W$). When W is exponentially large in the input length, $O(nW)$ is exponential. This is why SUBSET SUM is NP-complete even though it has a slick DP. Problem 7 implements the brute-force decision version.

5.5 Graph Coloring

Definition 5.9 (Graph k -Coloring). A k -coloring assigns each vertex one of k colors so that adjacent vertices get different colors. The k -COLORING problem asks whether a given graph admits one.

2-COLORING is in P (a graph is 2-colorable iff it is bipartite, checkable by BFS – Week 3). But 3-COLORING is NP-complete, via a reduction from 3-SAT using “true / false / base” triangle gadgets and per-clause OR-gadgets. The *chromatic number* $\chi(G)$ is the smallest k for which G is k -colorable; deciding $\chi(G) \leq 3$ is already hard. Problem 10 implements k -coloring by backtracking, and Problem 11 computes $\chi(G)$ by trying $k = 1, 2, 3, \dots$

6 The Structure of an NP-Completeness Proof

Every NP-completeness proof in this chapter follows the recipe of Corollary 4.5. It is worth isolating the moving parts so you can write your own:

1. **State the decision problem.** Make the yes/no question and the input encoding explicit. (Optimization problems must first be cast as decision problems with a threshold.)
2. **Prove membership in NP.** Identify a short certificate and a polynomial-time verifier. This is usually the easy half; do not skip it – without it you have only proven NP-hardness, not NP-completeness.
3. **Choose a source problem X .** Pick a known NP-complete problem that “feels structurally similar” to your target. 3-SAT is the universal default; VERTEX COVER or INDEPENDENT SET are natural for graph problems; SUBSET SUM for number/packing problems; HAMILTONIAN CYCLE for routing/ordering problems.
4. **Design the reduction (the gadget).** Give a polynomial-time map f from instances of X to instances of Y . This is the creative step: variables and clauses become vertices, paths, numbers, or colors.
5. **Prove correctness in both directions.** Show x is a yes-instance of $X \iff f(x)$ is a yes-instance of Y . Both directions are required; a one-directional argument is the most common mistake.
6. **Check polynomial time.** Confirm f runs in polynomial time and produces an instance of polynomial size.

A reduction is, at heart, a small program that translates the *logic* of one problem into the *structure* of another. Problem 12 makes this concrete by building an explicit mapping reduction f and verifying the equivalence “ $x \in A \iff f(x) \in B$ ” on random small instances – the experimental analogue of the both-directions proof.

6.1 A common pitfall: reducing the wrong way

To prove Y is hard, you must reduce a known-hard X to Y (i.e. $X \leq_p Y$), using Y ’s solver to solve X . A reduction the other way ($Y \leq_p X$) proves that Y is *easy* (no harder than X) and says nothing about Y ’s hardness. The direction of the arrow is everything: “hardness flows from X to Y .”

7 Summary and Looking Ahead

This week we changed our goal from *designing* efficient algorithms to *recognizing* when no efficient algorithm is likely to exist, and proving it. The key ideas:

- **Polynomial-time reductions** ($X \leq_p Y$) transfer tractability forward and intractability backward (Proposition 2.2); they compose (Lemma 4.4).
- **P, NP, certificates, verifiers.** P is efficient *solving*; NP is efficient *verifying* given a short certificate. $P \subseteq NP$; whether they are equal is the central open problem.
- **NP-completeness and Cook–Levin.** The NP-complete problems are the hardest in NP; Cook–Levin proves SAT/3-SAT is one, giving a fixed starting point.
- **The web of reductions.** $3\text{-SAT} \leq_p \text{INDEPENDENT SET} \leq_p \text{VERTEX COVER}$, $\text{INDEPENDENT SET} \leq_p \text{CLIQUE}$, and (via further gadgets) HAMILTONIAN CYCLE , TSP , SUBSET SUM , and 3-COLORING are all NP-complete.
- **The proof recipe:** state the decision problem, prove membership in NP, reduce a known NP-complete problem to it, prove both directions, check polynomial time.

Crucially, an NP-completeness result is not a dead end – it is a *redirection*. Knowing a problem is NP-complete tells you to stop looking for a polynomial exact algorithm and instead consider the techniques of the remaining weeks:

- **Week 13 – Coping with intractability.** Smarter exponential algorithms (like Held–Karp for TSP, Problem 9), **fixed-parameter tractable (FPT)** algorithms whose exponential blow-up is confined to a small parameter k (e.g. VERTEX COVER solvable in $O(2^k \cdot n)$ time), the larger complexity class **PSPACE** (problems solvable with polynomial memory but possibly exponential time, capturing games and quantified logic), and branch-and-bound search.
- **Week 14 – Approximation and local search.** Polynomial-time algorithms that return a provably near-optimal – though not exact – solution, trading a small guaranteed loss in quality for a huge gain in running time.

In short: this week teaches you to recognize the wall; the next two weeks teach you how to get over, around, or close to it.